ML E IA: DE DESARROLLO A PRODUCCIÓN · CONTENEDORES 2 DE 2

Del Dockerfile a producción

Cómo se escribe una imagen, por qué importa el orden de las líneas, qué no debe entrar y qué hace falta para que sea de producción. Con cinco prácticas y el laboratorio de roturas.

Dockerfile · Capas y caché · .dockerignore · Secretos · Usuario no root · Healthcheck

CÓMO USAR ESTAS DIAPOSITIVAS

Copia cada línea y pégala en la terminal de tu Codespace, **de una en una**, leyendo lo que sale.

Comandos y salidas **reales**: ejecutados con Docker 29.4.2 sobre Ubuntu 26.04.1 LTS (GitHub Actions, 10/11/2026). **Todavía no se han ejecutado en un Codespace del curso**: allí la versión de Docker, las columnas y los tiempos pueden diferir. Los identificadores largos se acortan con «…».

LA RECETA DE UNA IMAGEN

Un Dockerfile es una receta: una instrucción, una capa

```
FROM python:3.11-slim
WORKDIR /app
COPY app.py .
CMD ["python", "app.py"]
```

El del ejemplo 4: cuatro líneas, una imagen que funciona.

INSTRUCCIÓN	QUÉ HACE	CUÁNDO
`FROM`	Parte de una imagen base: un Debian 13 con Python 3.11	al construir
`WORKDIR`	Carpeta de trabajo dentro de la imagen (la crea si no existe)	al construir
`COPY`	Copia archivos del **contexto** (la carpeta del `.`) a la imagen	al construir
`CMD`	El comando que ejecuta el contenedor al arrancar	**al arrancar**

OTRAS QUE VERÁS HOY

`RUN`	Ejecuta un comando al construir (`pip install`)
`ENV`	Variable de entorno con un valor por defecto
`USER`	Usuario con el que corre el proceso
`EXPOSE`	Documenta el puerto; **no lo publica** (eso es `-p`)
`HEALTHCHECK`	Cómo comprobar que el contenedor está sano

LA IMAGEN ES UNA PILA

Cada instrucción añade una capa de solo lectura

DOCKER HISTORY SALUDO:1 (REAL, DE LA ÚLTIMA A LA PRIMERA)

`CMD ["python" "app.py"]`	0B
`COPY app.py .`	12.3kB
`WORKDIR /app`	8.19kB
`CMD ["python3"]`	0B
`RUN /bin/sh -c set -eux; for src in idle3 p…`	16.4kB

Dorado: las tuyas · azul: la imagen base (aquí solo se ven sus dos últimas). Las instrucciones que no tocan archivos (`CMD`) pesan 0 B.

QUÉ IMPLICA

- Una imagen = **una pila de capas**; el contenedor añade encima una fina de escritura.
- Las capas se **reutilizan** entre imágenes y entre construcciones.
- Casi todo el peso viene de la base; tus líneas añaden poco.

REUTILIZACIÓN, VISTA DE VERDAD

En el ejemplo 5, `WORKDIR /app` salió **CACHED**: esa capa ya existía de la imagen anterior.

```
#4 [1/4] FROM docker.io/library/python:3.11-
slim@sha256:07d4824036af…
#5 [2/4] WORKDIR /app
#5 CACHED
…
```

 EJECUTADO CON DOCKER REAL

Práctica A · Tu primera imagen

Construye una imagen de cuatro líneas, ejecútala y mira sus capas y su tamaño. Antes de construir, abre `Dockerfile` y `app.py` en el editor y léelos.

TERMINAL DEL CODESPACE

```
> cd "$(git rev-parse --show-toplevel)/01-Sesiones/S2-API-y-Docker/contenedores/ejemplos/04-primera-imagen"
> docker build --progress=plain -t saludo:1 . 2>&1 | grep -E "\[[0-9]+/[0-9]+\]|naming to"
> docker run --rm saludo:1
> docker history --format "{{.Size}}\t{{.CreatedBy}}" saludo:1 | head -5
> docker image ls python:3.11-slim
> docker image ls saludo:1
```

Selección de comandos; el ejemplo completo está en la guía práctica (ejemplo 4).

```
$ docker build --progress=plain -t saludo:1 . 2>&1 | grep -E "\[[0-9]…
#5 [1/3] FROM docker.io/library/python:3.11-slim@sha256:07d4824036af…
#6 [2/3] WORKDIR /app
#7 [3/3] COPY app.py .
#8 naming to docker.io/library/saludo:1 done
```

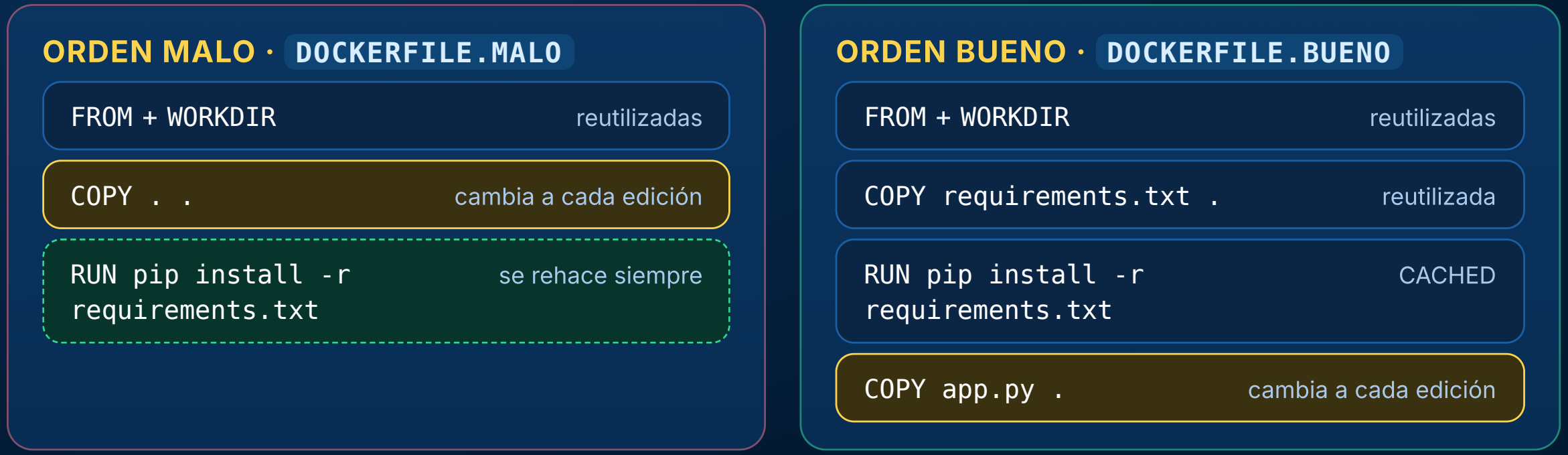
```
$ docker history --format "{{.Size}}\t{{.CreatedBy}}" saludo:1 | head -5
0B      CMD ["python" "app.py"]
12.3kB  COPY app.py .
8.19kB  WORKDIR /app
0B      CMD ["python3"]
16.4kB  RUN /bin/sh -c set -eux;  for src in idle3 p…
```

```
$ docker image ls saludo:1
IMAGE      ID             DISK USAGE   CONTENT SIZE   EXTRA
saludo:1   a16f43127cc8       187MB         45.6MB
```

PARA PENSAR · ¿Qué cambiarías para que imprima otro saludo? ¿Basta con editar `app.py` o hay que volver a construir?

LA REGLA DE ORO

Lo que casi nunca cambia, primero; lo que cambia siempre, al final



Azul: capa reutilizada · dorado: la capa que cambia · verde discontinuo: se rehace por arrastre.

Docker reutiliza una capa si **ella y todas las anteriores** no han cambiado; un cambio invalida esa capa y las siguientes. Es lo que hace el `Dockerfile` del proyecto: primero `COPY requirements-api.lock` y `pip install`, y después `COPY ia_mlops/` (`04-Proyecto-Base/deploy/docker/Dockerfile`).

 EJECUTADO CON DOCKER REAL

Práctica B · La caché de capas

Construye los dos Dockerfiles, cambia una línea del código y reconstruye. En nuestra ejecución, el orden malo tardó 8.0 s y el bueno 1.2 s (depende de la red y de la máquina: fíjate sobre todo en **CACHED**).

TERMINAL DEL CODESPACE

```
> cd "$(git rev-parse --show-toplevel)/01-Sesiones/S2-API-y-Docker/contenedores/ejemplos/05-cache"
> docker build --progress=plain -f Dockerfile.malo -t cache:malo . 2>&1 | grep -E "\[[0-9]+/[0-9]+\]|CACHED"
> echo "# cambio" >> app.py
> docker build --progress=plain -f Dockerfile.malo -t cache:malo . 2>&1 | grep -E "\[[0-9]+/[0-9]+\]|CACHED"
> docker build --progress=plain -f Dockerfile.bueno -t cache:bueno . 2>&1 | grep -E "\[[0-9]+/[0-9]+\]|CACHED"
> echo "# otro cambio" >> app.py
> docker build --progress=plain -f Dockerfile.bueno -t cache:bueno . 2>&1 | grep -E "\[[0-9]+/[0-9]+\]|CACHED"
```

Selección de comandos; el ejemplo completo está en la guía práctica (ejemplo 5).

```
$ docker build --progress=plain -f Dockerfile.malo -t cache:malo . 2>…
#4 [1/4] FROM docker.io/library/python:3.11-slim@sha256:07d4824036af…
#6 [2/4] WORKDIR /app
#6 CACHED
#7 [3/4] COPY . .
#8 [4/4] RUN pip install --no-cache-dir -r requirements.txt
```

```
$ docker build --progress=plain -f Dockerfile.bueno -t cache:bueno . …
#5 [1/5] FROM docker.io/library/python:3.11-slim@sha256:07d4824036af…
#6 [3/5] COPY requirements.txt .
#6 CACHED
#7 [2/5] WORKDIR /app
…
```

PARA PENSAR · En el `Dockerfile` del proyecto, ¿qué capa se reconstruye si cambias una línea de `ia_mlops/`? ¿Y si cambias `requirements-api.lock`?

QUÉ ENTRA EN LA IMAGEN SIN QUE LO PIENSES

`COPY . .` copia todo lo que haya en el contexto

Carpeta
todo lo que hay en el `.`

→

Contexto
Docker lo envía al motor de construcción

→

`COPY . .`
lo mete en la imagen

MEDIDO (REAL)

	SIN `.DOCKERIGNORE`	CON `.DOCKERIGNORE`
Contexto enviado	**52.44MB**	**178B**
Imagen (DISK USAGE)	**240MB**	**187MB**

Un archivo de 50 MB que no tiene nada que ver con la aplicación. **CONTENT SIZE** casi no cambia porque un archivo lleno de ceros se comprime hasta casi nada.

QUÉ GANAS CON UN BUEN `.DOCKERIGNORE`

- Imagen **más pequeña** y construcción **más rápida**.
- **No filtras secretos**: el del proyecto excluye `/.env`, `/*.key` y `**/credenciales*.json`.
- Tampoco entran `.git`, `data/`, `notebooks/` ni `tests/`.

 EJECUTADO CON DOCKER REAL

Práctica C · Contexto y .dockerignore

Crea un archivo de 50 MB, construye con él, añade un `.dockerignore` y vuelve a construir. Fíjate en «transferring context» y en el tamaño de la imagen.

TERMINAL DEL CODESPACE

```
> cd "$(git rev-parse --show-toplevel)/01-Sesiones/S2-API-y-Docker/contenedores/ejemplos/08-contexto"
> dd if=/dev/zero of=datos-grandes.bin bs=1M count=50 status=none && du -h datos-grandes.bin
> docker build --progress=plain -t contexto:con-basura . 2>&1 | grep -E "transferring context|COPY \. \."
> docker image ls contexto:con-basura
> printf 'datos-grandes.bin\n' > .dockerignore
> docker build --progress=plain -t contexto:limpio . 2>&1 | grep -E "transferring context|COPY \. \."
> docker image ls contexto:limpio
```

Selección de comandos; el ejemplo completo está en la guía práctica (ejemplo 8).

```
$ docker build --progress=plain -t contexto:con-basura . 2>&1 | grep …
#3 transferring context: 2B done
#5 transferring context: 52.44MB 0.1s done
#7 [3/3] COPY . .
```

```
$ docker build --progress=plain -t contexto:limpio . 2>&1 | grep -E "…
#3 transferring context: 58B done
#4 transferring context: 178B done
#7 [3/3] COPY . .
```

PARA PENSAR · Mira el `.dockerignore` de `04-Proyecto-Base/`. ¿Qué carpeta dejaría de ir en la imagen si la quitaras de ahí? ¿Y qué pasaría si `models/` estuviera en la lista?

VARIABLES DE ENTORNO

La configuración va al arrancar; el secreto, nunca en la imagen

BIEN · AL ARRANCAR

`-e NOMBRE=valor` o `--env-file archivo`: la misma imagen sirve para desarrollo y producción.

```
$ docker run --rm -e SALUDO="hola desde -e" -e
TOKEN_API=abc entorno:1
```

```
saludo: hola desde -e
token presente: True
```

```
$ docker run --rm --env-file mi.env entorno:1
```

```
saludo: desde archivo
token presente: True
```

MAL · `ENV TOKEN_API=…` EN EL DOCKERFILE

Construye sin error, pero queda grabado en las capas: quien tenga la imagen lo lee.

```
$ docker build -f Dockerfile.mal -t entorno:mal .
2>&1 | grep Secrets…
```

```
#1 WARN: SecretsUsedInArgOrEnv: Do not use ARG
or ENV instructions for sensitive data (ENV
"TOKEN_API") (line 6)
 - SecretsUsedInArgOrEnv: Do not use ARG or
ENV instructions for sensitive data (ENV
"TOKEN_API") (line 6)
```

```
$ docker history --no-trunc --format "
{{.CreatedBy}}" entorno:mal | g…
```

```
ENV TOKEN_API=no-es-un-secreto-real-123
```

Las claves reales se inyectan desde un **gestor de secretos** (en GCP, Secret Manager) y llegan al contenedor como variables o archivos al arrancar. Eso se verá en S3. DOCUMENTACIÓN · SIN PROBAR AQUÍ

 EJECUTADO CON DOCKER REAL

Práctica D · Configuración y secretos

Pasa variables con `-e` y con un archivo, y comprueba con tus ojos que un `ENV` con un «secreto» queda a la vista en el historial de la imagen.

TERMINAL DEL CODESPACE

```
> cd "$(git rev-parse --show-toplevel)/01-Sesiones/S2-API-y-Docker/contenedores/ejemplos/06-entorno"
> docker build -q -t entorno:1 .
> docker run --rm entorno:1
> docker run --rm -e SALUDO="hola desde -e" -e TOKEN_API=abc entorno:1
> printf 'SALUDO=desde archivo\nTOKEN_API=xyz\n' > mi.env
> docker run --rm --env-file mi.env entorno:1
> docker build -f Dockerfile.mal -t entorno:mal . 2>&1 | grep SecretsUsedInArgOrEnv
> docker history --no-trunc --format "{{.CreatedBy}}" entorno:mal | grep TOKEN_API
> rm -f mi.env
```

Selección de comandos; el ejemplo completo está en la guía práctica (ejemplo 6).

```
$ docker run --rm -e SALUDO="hola desde -e" -e TOKEN_API=abc entorno:1
```

```
saludo: hola desde -e
token presente: True
```

```
$ docker run --rm --env-file mi.env entorno:1
```

```
saludo: desde archivo
token presente: True
```

```
$ docker history --no-trunc --format "{{.CreatedBy}}" entorno:mal | g…
```

```
ENV TOKEN_API=no-es-un-secreto-real-123
```

PARA PENSAR · El `.dockerignore` del proyecto excluye `/.env`, `/*.key` y `**/credenciales*.json`. ¿Qué evita eso exactamente?

MÁS QUE «QUE ARRANQUE»

Lo que distingue una imagen de producción

USUARIO SIN PRIVILEGIOS

`USER 10001` : si alguien explota la aplicación, tiene menos margen.

```
Sin USER:
el proceso corre con el usuario numero 0
Con USER 10001:
el proceso corre con el usuario numero 10001
```

COMPROBACIÓN DE SALUD

`HEALTHCHECK` : Docker (o la nube) sabe si responde.

```
NAMES     STATUS                    PORTS
mini      Up 6 seconds (healthy)    0.0.0.0:8081-
>8080/tcp, [::]:8081->8080/tcp
```

REGISTROS POR PANTALLA

JSON a stdout, una línea por petición: `docker logs`.

```
{"ts": …, "evento": "arranque", "puerto": 8080,
"modelo": "v2"}
{"ts": …, "metodo": "GET", "ruta": "/salud",
"codigo": 200}
…
```

LÍMITES DE RECURSOS

`--memory 128m`. Si el proceso se pasa, lo matan: `137` (rotura 4).

```
NAME      MEM USAGE / LIMIT   MEM %
mini      13.97MiB / 128MiB   10.91%
```

 EJECUTADO CON DOCKER REAL

Práctica E · Una API de producción en miniatura

Una API de 40 líneas con usuario sin privilegios, `HEALTHCHECK`, registros JSON y límite de memoria. Espera ~6 s tras el `docker run` para ver `healthy` (antes sale `health: starting`).

TERMINAL DEL CODESPACE

```
> cd "$(git rev-parse --show-toplevel)/01-Sesiones/S2-API-y-Docker/contenedores/ejemplos/09-api-minima"
> docker build -q -t api-minima:1 .
> docker run -d --name mini -p 8081:8080 -e MODELO=v2 --memory 128m api-minima:1
> docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
> docker logs mini
> docker stats --no-stream --format "table {{.Name}}\t{{.MemUsage}}\t{{.MemPerc}}" mini
> docker exec mini id -u
> docker rm -f mini
```

Selección de comandos; el ejemplo completo está en la guía práctica (ejemplo 9).

```
$ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES     STATUS                    PORTS
mini      Up 6 seconds (healthy)    0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
```

```
$ docker logs mini
{"ts": …, "evento": "arranque", "puerto": 8080, "modelo": "v2"}
{"ts": …, "metodo": "GET", "ruta": "/salud", "codigo": 200}
{"ts": …, "metodo": "GET", "ruta": "/modelo", "codigo": 200}
{"ts": …, "metodo": "GET", "ruta": "/nada", "codigo": 404}
```

```
$ docker stats --no-stream --format "table {{.Name}}\t{{.MemUsage}}\t…
NAME      MEM USAGE / LIMIT   MEM %
mini      13.97MiB / 128MiB   10.91%
```

PARA PENSAR · Compara este `Dockerfile` con `04-Proyecto-Base/deploy/docker/Dockerfile`: ¿qué tienen igual y qué añade el del proyecto?

La imagen de la API del curso, línea a línea

EN EL DOCKERFILE DEL PROYECTO	PARA QUÉ	DÓNDE LO HAS VISTO
`FROM python:3.11-slim`	La misma base que los ejemplos	Ejemplos 1 y 4
`ENV PYTHONUNBUFFERED=1 …`	Ajustes de Python y de pip y la ruta del modelo; ninguno es un secreto	Ejemplo 6
`COPY requirements-api.lock .` + `RUN pip install -r …`	Dependencias **primero**: la capa solo se rehace si cambian las versiones	Ejemplo 5
`COPY ia_mlops/ …` + `COPY models/modelo_mlflow/ …`	Solo el código y el modelo necesarios; el modelo viaja **dentro** de la imagen	Ejemplos 7 y 8 · rotura 3
`.dockerignore`	Deja fuera datos, cuadernos, pruebas, `.git` y claves	Ejemplo 8
`RUN useradd --uid 10001 …` + `USER 10001`	No se ejecuta como root	Ejemplos 8 y 9
`EXPOSE 8080` + `HEALTHCHECK --interval=30s …`	Puerto de la API y comprobación de salud (en el ejemplo, cada 3 s para verlo en vivo)	Ejemplo 9
`CMD ["uvicorn", … "--host", "0.0.0.0", "--port", "8080"]`	Proceso principal; escucha en todas las interfaces	Ejemplo 3

Lo que Google documenta del contenedor en Cloud Run

DOCUMENTACIÓN · SIN PROBAR AQUÍ

TEMA	QUÉ DICE LA DOCUMENTACIÓN DE GOOGLE	CON LO QUE HAS PRACTICADO
Puerto y dirección	Debe escuchar en `0.0.0.0`, en el puerto al que se envían las peticiones (**8080** por defecto, configurable). Cloud Run inyecta la variable `PORT`.	Ejemplo 3 · la API del curso usa 8080 y 0.0.0.0
Memoria	Si se supera el límite configurado, la instancia se termina.	Rotura 4 · código 137
Sistema de archivos	Está en memoria y lo escrito no persiste cuando la instancia se detiene.	Ejemplos 2 y 7 · lo escrito dentro se pierde
Secretos y registros	No consultado en este material.	Se verá en S3 (Secret Manager y registros)

Fuente: https://docs.cloud.google.com/run/docs/container-contract (consultada el 10/11/2026; resumida con una herramienta de lectura). **No se ha desplegado nada en Cloud Run en este material**: en S3 lo comprobamos de verdad.

≈34 MIN · LABORATORIO-ROMPER.MD

Laboratorio: cinco fallos que vas a provocar y arreglar

ROTURA	SÍNTOMA	PRIMER COMANDO	TIEMPO
1. El puerto ya está ocupado	Arrancas un segundo contenedor y Docker devuelve un error.	`docker ps -a --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"`	5 min
2. El contenedor se para nada más arrancar	`docker run` termina al instante y no hace lo que esperas.	`docker ps -a --format "table {{.Names}}\t{{.Status}}"`	8 min
3. La construcción falla: falta un archivo	`docker build` se detiene con un error de `COPY`.	`ls`	8 min
4. El contenedor muere por falta de memoria	El proceso desaparece sin mensaje de error y el código de salida es 137.	`docker ps -a --format "table {{.Names}}\t{{.Status}}"`	8 min
5. «Arranca» pero no está en marcha	`docker run -d` devuelve un identificador, pero `docker ps` no muestra nada.	`docker ps -a --format "table {{.Names}}\t{{.Status}}"`	5 min

CÓMO SE TRABAJA

- **Provocas** el fallo y lo **diagnosticas** con Docker.
- Escribes tu **hipótesis** antes de ver el arreglo.
- Abres el **arreglo** y comparas.

QUÉ TE LLEVAS

Un método para cuando un contenedor no hace lo que esperas, y que las cinco roturas comparten. Los comandos y sus salidas reales están en `laboratorio-romper.md`.

EL MISMO EN LAS CINCO ROTURAS

Cuando algo falla: cuatro preguntas, siempre en este orden

1 · ¿En marcha? `docker ps -a` → **2 · ¿Qué dijo?** `docker logs x` → **3 · ¿Cómo está?** `docker inspect x` → **4 · ¿Qué hay dentro?** `docker exec x …`

CÓDIGOS DE SALIDA QUE HAS VISTO

CÓDIGO	SIGNIFICA	DÓNDE
`0`	El proceso terminó bien (a veces, demasiado pronto: rotura 5)	Ejemplo 1 · rotura 5
`2`	El programa falló (aquí, no encontró `aap.py`)	Rotura 2
`125`	`docker run` no pudo arrancar el contenedor (aquí, puerto ocupado)	Rotura 1
`137`	128 + 9: terminado a la fuerza (un `stop` que no atiende, o falta de memoria)	Ejemplo 2 · rotura 4
`1`	En `docker build`: la construcción falló (aquí, un `COPY` sin archivo)	Rotura 3

Cambia **una** cosa cada vez, repite y comprueba.

Listo para el laboratorio

- [ ] He construido una imagen y he mirado sus capas. — Práctica A
- [ ] Sé ordenar un Dockerfile para aprovechar la caché y lo he visto con `CACHED`. — Práctica B
- [ ] He usado `.dockerignore` y he visto cambiar el contexto y el tamaño. — Práctica C
- [ ] Paso configuración con `-e` y sé por qué un secreto no va en `ENV`. — Práctica D
- [ ] He visto una imagen con usuario sin privilegios, `HEALTHCHECK`, registros y límite de memoria. — Práctica E
- [ ] Conozco el método de diagnóstico: `ps -a` → `logs` → `inspect` → `exec`. — Laboratorio

Comprueba cada punto antes de pasar a lo siguiente.

Siguiente: Laboratorio «romperlo a propósito» →